Functional Dependencies

Introduction to database management system

Closure

- The Closure Of Functional Dependency means the complete set of all possible attributes that can be functionally derived from given functional dependency using the inference rules known as Armstrong's Rules.
- If "F" is a functional dependency then closure of functional dependency can be denoted using "$\{F\}^{+}$".
- There are three steps to calculate closure of functional dependency.

Closure

- Step-1 : Add the attributes which are present on Left Hand Side in the original functional dependency.
- Step-2 : Now, add the attributes present on the Right Hand Side of the functional dependency.
- Step-3 : With the help of attributes present on Right Hand Side, check the other attributes that can be derived from the other given functional dependencies. Repeat this process until all the possible attributes which can be derived are added in the closure.

Closure Property

- R={A,B,C,D}
- FD ={A → B, B → C, C → D, }
- $A^+ = ABCD$
- $B^+ = BCD$
- $C^+ = CD$
- $D^+ = D$
- **Candidate Key = {A}**
- **Prime Attribute = {A}**
- **Non Prime Attribute(Non key Attribute) = {B,C,D}**

Closure Property

- R={A,B,C,D}
- FD ={A → B, B → C, C → D, D → A }
- $A^+ = ABCD$
- $B^+ = BCDA$
- $C^+ = CDAB$
- $D^+ = DABC$
- **Candidate Key = {A,B,C,D}**
- **Prime Attribute = {A,B,C,D}**
- **Non Prime Attribute(Non key Attribute) = {Φ}**

Closure Property

- R={A,B,C,D, E}
- FD ={A →BC,C →B, D →E, E →D }
- $A^+ = ABC$
- $B^+ = B$
- $C^+ = CB$
- $D^+ = D$E
- $E^+ =$ED
- $AB^+ = ABC$
- $AC^+ = ABC$
- $AD^+ = ABCDE$
- $AE^+ = ABCED$
- $BC^+ = BC$
- $BD^+ = BDE$
- $BE^+ = BED$
- $CB^+ = BC$
- $CD^+ = BCDE$
- $CE^+ = BCDE$
- $DE^+ = DE$
- ……

Closure Property

- **Candidate Key = {AD,AE}**
- **Prime Attribute = {A,D,E}**
- **Non Prime Attribute(Non key Attribute) = {**B,C**}**

Minimal Cover

- A minimal cover is simplified and reduced version of given set of functional dependencies
- Since it is a reduced version, it is also called as **Irreducible set**. It is also called as **Canonical Cover**.
- **Steps to Find Minimal Cover**

1. **Split the right-hand attributes of all FDs.**
2. **Remove all redundant FDs.**
3. **Find the Extraneous attribute and remove it.**

Minimal Cover

- R={A,B,C,D}
- FD ={A → B, C → B, D → ABC, AC → D }
- **Step1:Split the right-hand attributes of all FDs.**
  - A → B, C → B, D → A, D → B, D → C, AC → D
- **Step2: Remove all redundant FDs**
  - A → B,
  - C → B,
  - D → A,
  - D → C,
  - AC → D
- **Step3:Find the Extraneous attribute and remove it.**
- AC → D (you can remove A by taking C+ if A comes in C+, and can remove C by taking A+ if C comes in A+)
- A → B, C → B, D → A, D → C, AC → D or A → B, C → B, D → AC, AC → D

- A^+=A
- C^+=C
- D^+=BC
- D^+=AB
- D^+=AB
- AC^+=D

Minimal Cover Example

- **1) Split the right-hand attributes of all FDs.**
  Example
  A->XY => A->X, A->Y
- **2) Remove all redundant FDs.**
  Example
  { A->B, B->C, A->C }
  Here A->C is redundant since it can already be achieved using the Transitivity Property.
- **3) Find the Extraneous attribute and remove it.**
  Example
  AB->C, either A or B or none can be extraneous.
  If A closure contains B then B is extraneous and it can be removed.
  If B closure contains A then A is extraneous and it can be removed.

Minimal Cover Example

- **Example 1**
  Minimize {A->C, AC->D, E->H, E->AD}
- **Step 1**: {A->C, AC->D, E->H, E->A, E->D}
- **Step 2**: {A->C, AC->D, E->H, E->A}
  Here Redundant FD : {E->D}
- **Step 3**: {AC->D}
  {A}+ = {A,C}
  Therefore C is extraneous and is removed.
  {A->D}
- Minimal Cover = {A->C, A->D, E->H, E->A}

Minimal Cover Example

- **Example 2**
  Minimize {AB->C, D->E, AB->E, E->C}
- **Step 1**: {AB->C, D->E, AB->E, E->C}
- **Step 2**: {D->E, AB->E, E->C}
  Here Redundant FD = {AB->C}
- **Step 3**: {AB->E}
  {A}+ = {A}
  {B}+ = {B}
  There is no extraneous attribute.
- Therefore, Minimal cover = {D->E, AB->E, E->C}

Equivalence of Functional Dependencies

- Two or more than two sets of functional dependencies are called equivalence if the right-hand side of one set of functional dependency can be determined using the second FD set, similarly the right-hand side of the second FD set can be determined using the first FD set.

Example 1

- X= {A → B, B → C} and Y= {A → B, B → C, A → C}
- First we will check if X covers Y
- After that we will check if Y covers X
- Sol: We will check first if X covers Y or not
- So, we will take Y first Dependency and take Closure from X
- A^+=ABC --- This satisfy the both dependencies of Y {A → B, A → C}
- B^+=BC --- This satisfy the second dependencies of Y {B → C}
- This mean X covers Y

Example 1

- X= {A → B, B → C} and Y= {A → B, B → C, A → C}
- Now We will check if Y covers X or not
- So, we will take X first Dependency and take Closure from Y
- A^+=ABC --- This satisfy the dependencies of X {A → B,}
- B^+=BC --- This satisfy the second dependencies of X {B → C}
- This mean Y covers X
- Hence X and Y are equivalent.

Example 2

- X= {AB → CD, B → C, C → D} and Y= {AB → C, AB → D, C → D}
- First we will check if X covers Y
- After that we will check if Y covers X
- Sol: We will check first if X covers Y or not
- So, we will take Y first Dependency Closure from X
- AB^+=ABCD - This satisfy the both dependencies of Y {AB → C, AB → D}
- C^+=CD - This satisfy the second dependencies of Y {C → D}
- This mean X covers Y

Example 2

- X= {AB → CD, B → C, C → D} and Y= {AB → C, AB → D, C → D}
- Now we will check if Y covers X
- So, we will take X first Dependency and take Closure from Y
- AB^+=ABCD - This satisfy the both dependencies of Y {AB → CD}
- B^+=B – This not satisfying the second dependencies of Y {B → C}
- C^+=CD – This is satisfying the third dependencies of Y {C → D}
- Hence X and Y are not equivalent.

Example 3

- X= {A →BC, B →CDE, AE →F} and Y= {A →BCF, B →DE, E →AB}
- First we will check if X covers Y
- After that we will check if Y covers X
- Sol:
- A^+={A,B,C,D,E,F} contains B,C,F
- B^+={B,C,D,E} contains D,E
- E^+={E} contains A,B
- So X does not cover Y.
- Hence X and Y are not equivalent.

Example 4

- X= {A →C, AC →D, E →AD, E →H,} and Y= {A →CD, E →AH}
- First we will check if X covers Y
- After that we will check if Y covers X
- Sol:
- A^+={A,C,D} contains C,D
- E^+={A,D,E,H} contains A,H
- So X covers Y

Example 4

- X= {A →C, AC →D, E →AD, E →H,} and Y= {A →CD, E →AH}

- To check Y covers X:
- A^+={A,C,D} contains C
- { A,C}+ ={A,C,D} contains D
- E^+={A,C,D,E,H} contains A,D,H
- So Y also covers X.
- Hence X and Y are equivalent.

Determines Normal forms from FD

- **Steps to find the highest normal form of relation:**

1. Find all possible candidate keys of the relation.
2. Divide all attributes into two categories: prime attributes and non-prime attributes.
3. Check for 1st normal form then 2nd and so on. If it fails to satisfy the nth normal form condition, the highest normal form will be n-1.

- **Partial Dependency –** If the proper subset of candidate key determines non-prime attribute, it is called partial dependency.

Determines Normal forms from FD

- Every FD is 1st normal Form because RDBMS don't allowed multivalued and composite attribute
- For 2nd normal form- No Partial Dependency
  - When Left side is superkey
  - Left side should not be proper subset of Candidate key which determines non prime key attribute
- For 3rd normal form- No Transitive dependency
  - When Left side should be Superkey
  - When Left side is not be superkey then right side should be prime attribute.

Example 1

- Find the highest normal form of a relation
- R(A,B,C,D,E)
- FD ={BC->D, AC->BE, B->E}
- Step 1. As we can see, (AC)+ ={A,C,B,E,D} but none of its subset can determine all attribute of relation, So AC will be candidate key. A or C can’t be derived from any other attribute of the relation, so there will be only 1 candidate key {AC}.

Example 1

R(A,B,C,D,E)

FD ={BC->D, AC->BE, B->E}

- Step 2. Prime attributes are those attributes that are part of candidate key {A, C} in this example and others will be non-prime {B, D, E} in this example.

- Step 3. The relation R is in 1st normal form as a relational DBMS does not allow multi-valued or composite attribute.
- The relation is in 2nd normal form because BC->D is in 2nd normal form (BC is not a proper subset of candidate key AC) and AC->BE is in 2nd normal form (AC is candidate key) and B->E is in 2nd normal form (B is not a proper subset of candidate key AC).
- The relation is not in 3rd normal form because in BC->D (neither BC is a super key nor D is a prime attribute) and in B->E (neither B is a super key nor E is a prime attribute) but to satisfy 3rd normal for, either LHS of an FD should be super key or RHS should be prime attribute.
  So the highest normal form of relation will be 2nd Normal form.

Example 2

- **Find the highest normal form of a relation**
- R(A,B,C,D,E)
- FD = {A->D, B->A, BC->D, AC->BE}
- Solution
- **Step 1:** two candidate keys {AC, BC}.
- **Step 2.** The prime attribute {A, B, C}
- non-prime {D, E}

Example 2

R(A,B,C,D,E)

FD = {A->D, B->A, BC->D, AC->BE}

- **Step 3.** The relation R is in 1st normal form as a relational DBMS does not allow multi-valued or composite attributes.
- The relation is not in the 2nd Normal form because A->D is partial dependency (A which is a subset of candidate key AC is determining non-prime attribute D) and the 2nd normal form does not allow partial dependency.

Example 3

- Find the highest normal form of a relation
- **R(A,B,C,D,E)**
- **FD ={BC->D, AC->BE, B->E}**
- Solution:
- Step 1: candidate key {AC}.
- **Step 2.** The prime attribute {A,C}
- non-prime {B,D,E} in this example.

Example 3

R(A,B,C,D,E)

FD ={BC->D, AC->BE, B->E}

- **Step 3.** The relation R is in 1st normal form as a relational DBMS does not allow multi-valued or composite attributes.
- The relation is in 2nd normal form because BC->D is in 2nd normal form (BC is not a proper subset of candidate key AC) and AC->BE is in 2nd normal form (AC is candidate key) and B->E is in 2nd normal form (B is not a proper subset of candidate key AC).
- The relation is not in 3rd normal form because in BC->D (neither BC is a super key nor D is a prime attribute) and in B->E (neither B is a super key nor E is a prime attribute) but to satisfy 3rd normal for, either LHS of an FD should be super key or RHS should be a prime attribute.
- So the highest normal form of relation will be the 2nd Normal form.